



南开大学
Nankai University

南开大学

计算机学院和密码与网络空间安全学院

《数据安全》课程作业

秘密共享实践

姓名：周重天

学号：2311082

专业：计算机科学与技术

2026 年 4 月 28 日

目录

1	实验要求	2
2	实验原理	2
2.1	秘密共享	2
2.2	门限秘密共享	2
2.3	基于 Shamir 门限秘密共享的协议设计	3
3	实验环境	3
4	实验过程	4
5	实验心得与体会	10

1 实验要求

假设有三个同学需要对班里的优秀干部 Alice、Bob、Charles、Douglas 进行投票，最后统计各个班干部获得的票数。这个时候就可以利用 Shamir 秘密共享将各个投票方的投票分享出去并进行隐私求和计算。

借鉴实验 3.2，实现三个人对于他们拥有的数据的平均值的计算。

2 实验原理

2.1 秘密共享

秘密共享 (Secret Sharing) 是一种将秘密分割存储的密码技术，目的是阻止秘密过于集中，以达到分散风险和容忍入侵的目的，是信息安全和数据保密中的重要手段。目前，秘密共享已成为一种重要密码学工具，在诸多多方安全计算协议中被使用，例如拜占庭协议、多方隐私集合求交协议、阈值密码学等。

基于秘密分配和运算的形式，我们将秘密共享分为基于位运算的加性秘密共享和基于线性代数的线性秘密共享。依据秘密重构的条件或者秘密分享的份额数量等，又可以将秘密共享分为如下类别：

- **门限秘密共享**：任意大于等于阈值的参与方集合可重构出秘密；
- **多重秘密共享**：参与方的子秘密可以多次使用，分别恢复多个共享秘密；
- **多秘密共享**：一次共享，共享多个秘密，且子秘密可以重复使用；
- **可验证秘密共享**：可通过公共变量验证自己子秘密的正确性；
- **动态秘密共享**：允许添加或删除参与方，定期或不定期更新参与方的子秘密，还允许在不同的时间恢复不同的秘密。

2.2 门限秘密共享

门限秘密共享 (Threshold Secret Sharing) 是一种密码学技术，可以将秘密分割成多个部分，并将这些部分分配给不同的参与者，从而保护秘密免受单点故障和恶意攻击。

在门限秘密共享中，将秘密分成 n 个部分，其中只要 k 个部分被收集，就可以重构出原始秘密。这个 k 值称为“门限”，通常满足 $k \leq n$ ，以确保秘密不被泄露。只有在收集到至少 k 个部分时，才能重构出秘密，因此，即使一些部分被泄露或丢失，也可以保持秘密的安全性。

门限秘密共享可以用于许多场景，例如：多方拥有的敏感信息（例如金融账户密码）、公共密钥基础设施、加密密钥管理等等。

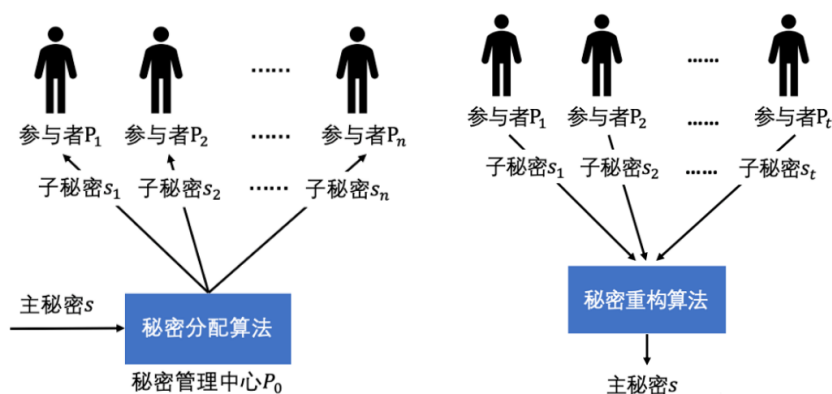


图 2.1: 门限秘密共享实验原理

2.3 基于 Shamir 门限秘密共享的协议设计

首先，要基于 Shamir 门限秘密共享进行协议设计。

以计算 Alice 的票数为例，三个同学的隐私输入为 0 或 1，0 表示不投票给 Alice，1 表示投票给 Alice。假设三个同学 $Student_1$ 、 $Student_2$ 和 $Student_3$ 分别拥有隐私输入 a 、 b 和 c ，他们将各自的隐私输入通过 (2,3) 门限的 Shamir 秘密共享分享给另外两个同学： $Student_1$ 获得 a 、 b 和 c 的秘密份额 a_1 、 b_1 和 c_1 ； $Student_2$ 获得秘密份额 a_2 、 b_2 和 c_2 ； $Student_3$ 获得秘密份额 a_3 、 b_3 和 c_3 。之后，三位同学各自将获得的秘密份额相加，分别得到 $d_1 = a_1 + b_1 + c_1$ ， $d_2 = a_2 + b_2 + c_2$ 和 $d_3 = a_3 + b_3 + c_3$ 。一个计票员从三个同学中任选两个，例如 $Student_2$ 和 $Student_3$ ，获得他们拥有的 d_2 和 d_3 ，就可以重构出 $d = a + b + c$ ，也就是 Alice 获得的票数总和。

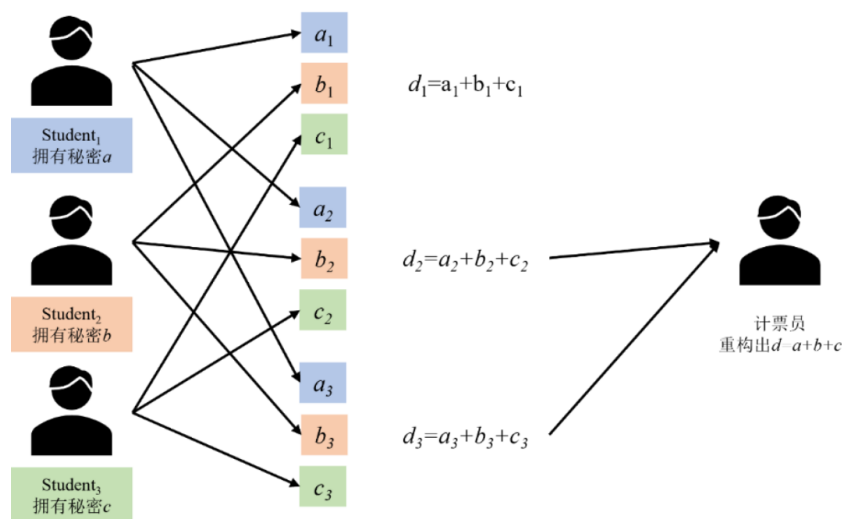


图 2.2: 实验具体原理

3 实验环境

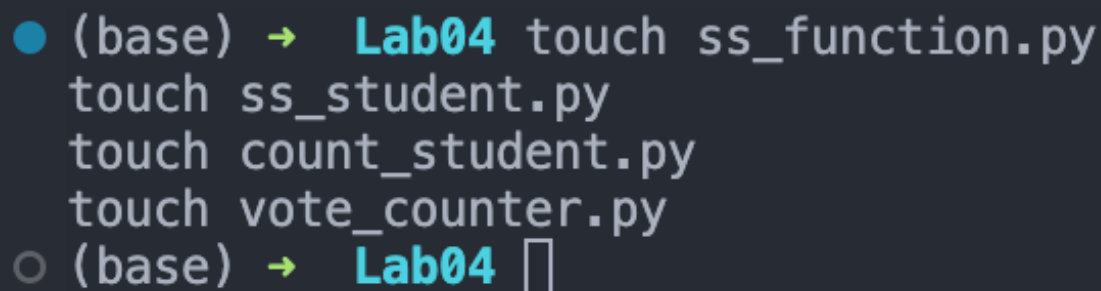
本次实验在本机的 macos 上完成，仅涉及到 python 及相关环境包。

4 实验过程

首先，我们在 Lab4 目录下新建一个文件夹 vote，在 vote 文件夹下打开终端，分别输入以下命令行：

```
1 touch ss_function.py
2 touch ss_student.py
3 touch count_student.py
4 touch vote_counter.py
```

执行完毕后，如下所示，我们成功新建了四个 python 文件。



```
● (base) → Lab04 touch ss_function.py
touch ss_student.py
touch count_student.py
touch vote_counter.py
○ (base) → Lab04 □
```

图 4.3: 新建四个文件

然后，将如下代码复制到 ss_function.py，ss_function.py 代码中定义了一些秘密共享过程中三个学生以及计票员会用到的函数。

具体代码如下所示：

```
1 import random
2
3 # 快速幂计算  $a^b \pmod p$ 
4 def quickpower(a,b,p):
5     a=a%p
6     ans=1
7     while b!=0:
8         if b&1:
9             ans=(ans*a)%p
10            b>>=1
11            a=(a*a)%p
12    return ans
13
14 # 构建多项式:  $x_0$  为常数项系数,  $T$  为最高次项次数,  $p$  为模数,  $fname$  为多项式名
15 def get_polynomial(x0,T,p,fname):
16     f=[]
17     f.append(x0)
18     for i in range(0,T):
```

```
19         f.append(random.randrange(0,p))
20     # 输出多项式
21     f_print='f'+fname+'='+str(f[0])
22     for i in range(1,T+1):
23         f_print+='+'+str(f[i])+'x^'+str(i)
24     print(f_print)
25     return f
26
27 # 计算多项式值
28 def count_polynomial(f,x,p):
29     ans=f[0]
30     for i in range(1,len(f)):
31         ans=(ans+f[i]*quickpower(x,i,p))%p
32     return ans
33
34 # 重构函数 f 并返回 f(0)
35 def restructure_polynomial(x,fx,t,p):
36     ans=0
37     # 利用多项式插值法计算出 x=0 时多项式的值
38     for i in range(0,t):
39         fx[i]=fx[i]%p
40         fxi=1
41         # 在模 p 下, (a/b)%p=(a*c)%p, 其中 c 为 b 在模 p 下的逆元, c=b^(p-2)%p
42         for j in range(0,t):
43             if j !=i:
44                 fxi=(-1*fxi*x[j]*quickpower(x[i]-x[j],p-2,p))%p
45         fxi=(fxi*fx[i])%p
46         ans=(ans+fxi)%p
47     return ans
```

然后将如下代码复制到 ss_student.py, 三个学生分别执行 ss_student.py, 将自己的秘密投票值共享给另外两个学生。

具体代码如下所示:

```
1 import ss_function as ss_f
2 # 设置模数 p
3 p=1000000007
4 print(f'模数 p: {p}')
5 # 输入参与方 id 以及秘密 s
6 id=int(input("请输入参与方 id:"))
7 s=int(input(f'请输入 student_{id}的投票值 s:'))
8 # 秘密份额为 (share_x,share_y)
9 shares_x=[1,2,3]
```

```
10 shares_y=[]
11 # 计算多项式及秘密份额 (t=2,n=3)
12 print(f'Student_{id}的投票值的多项式及秘密份额: ')
13 f=ss_f.get_polynomial(s,1,p,str(id))
14 temp=[]
15 for j in range(0,3):
16     temp.append(ss_f.count_polynomial(f,shares_x[j],p))
17     print(f'({shares_x[j]},{temp[j]})')
18     shares_y.append(temp[j])
19 #Student_id 将自己的投票值的秘密份额分享给另外两个学生
20 # 将三份秘密份额分别保存到 student_id_1.txt,student_id_2.txt,student_id_3.txt
21 #Student_i 获得 Student_id_i.txt
22 for i in range(1,4):
23     with open(f'student_{id}_{i}.txt','w') as f:
24         f.write(str(shares_y[i-1]))
```

然后在命令行执行下面的命令:

```
1 python3 ss_student.py
2 1
3 0
4 python3 ss_student.py
5 2
6 1
7 python3 ss_student.py
8 3
9 0
```

结果如下所示:

然后，将以下代码复制到 `count_student.py`，三个学生分别执行 `count_student.py`，获取另外两个学生的投票值的秘密份额，并将三个投票值的秘密份额相加。

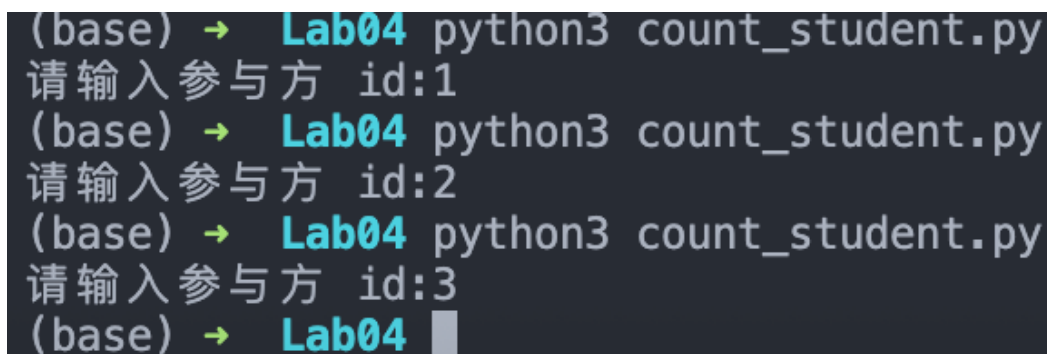
具体代码如下所示：

```
1 p=1000000007
2 # 输入参与方 id
3 id=int(input("请输入参与方 id:"))
4 #Student_id 读取属于自己的秘密份额 student_1_id.txt, student_2_id.txt, student_3_id.txt
5 data=[]
6 for i in range(1,4):
7     with open(f'student_{i}_{id}.txt', "r") as f: # 打开文本
8         data.append(int(f.read())) # 读取文本
9 # 计算三个秘密份额的和
10 d=0
11 for i in range(0,3):
12     d=(d+data[i])%p
13 # 将求和后的秘密份额保存到文件 d_id.txt 内
14 with open(f'd_{id}.txt','w') as f:
15     f.write(str(d))
```

执行下面三个语句，把三个投票值的秘密份额相加，分别保存到三个文件中。

```
1 python3 count_student.py
2 1
3 python3 count_student.py
4 2
5 python3 count_student.py
6 3
```

执行完毕后，结果如下所示：



```
(base) -> Lab04 python3 count_student.py
请输入参与方 id:1
(base) -> Lab04 python3 count_student.py
请输入参与方 id:2
(base) -> Lab04 python3 count_student.py
请输入参与方 id:3
(base) -> Lab04
```

图 4.6: 执行三个命令完成求和

结果如下所示，在文件夹 `vote` 下会产生 3 个 `txt` 文件，分别为 `d_1.txt`、`d_2.txt` 和 `d_3.txt`。

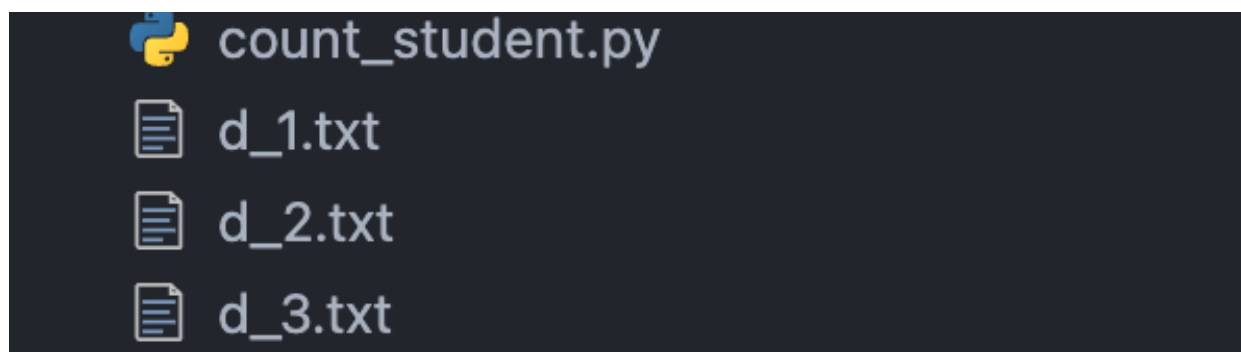


图 4.7: 执行三个命令完成求和

继续，将以下代码复制到 `vote_counter.py`，具体代码如下所示：

```
1 import ss_function as ss_f
2 # 设置模数 p
3 p=1000000007
4 # 随机选取两个参与方，例如 student2 和 student3，获得 d2,d3，从而恢复出 d=a+b+c
5 # 读取 d2,d3
6 d_23=[]
7 for i in range(2,4):
8     with open(f'd_{i}.txt', "r") as f: # 打开文本
9         d_23.append(int(f.read())) # 读取文本
10 # 加法重构获得 d
11 d=ss_f.restructure_polynomial([2,3],d_23,2,p)
12 # 这一步是为了计算平均值
13 d = d / 3
14 print(f'得票结果为: {d}')
```

执行命令：

```
1 python3 vote_counter.py
```

结果如下所示：

```
(base) → Lab04 python3 ss_student.py
模数 p: 1000000007
请输入参与方 id:3
请输入 student_3的投票值 s:0
Student_3的投票值的多项式及秘密份额:
f3=0+78005694x^1
(1,78005694)
(2,156011388)
(3,234017082)
(base) → Lab04 python3 count_student.py
请输入参与方 id:1
(base) → Lab04 python3 count_student.py
请输入参与方 id:2
(base) → Lab04 python3 count_student.py
请输入参与方 id:3
(base) → Lab04 python3 vote_counter.py
得票结果为: 0.3333333333333333
(base) → Lab04
```

图 4.8: 计算平均值的结果

由此可见，三人的原始投票为 0、1、0，计算得到的平均值为 0.333，说明平均值计算正确。这样的话，实验就成功了。

5 实验心得与体会

通过本次实验，我进一步理解了 Shamir 门限秘密共享在隐私计算中的应用方式。实验中，三位参与方并没有直接公开自己的原始输入，而是先将投票值构造成一次多项式上的秘密份额，再分别分发给其他参与方。随后，各参与方只对自己持有的份额进行本地求和，计票方只需收集其中两个求和后的份额，就可以利用拉格朗日插值恢复出总和，并进一步得到平均值。整个过程体现了秘密共享的线性性质，也说明在不暴露单个参与方隐私数据的情况下，仍然可以完成有效的统计计算。

此外，三人的输入为 0,1,0，最终恢复出的平均值为 0.333，与理论计算结果一致，说明程序实现是正确的。实验过程中我也认识到，模数的选择、多项式构造、份额生成以及插值重构都是协议正确运行的关键步骤。如果份额读取错误或重构时选取的参与方数量不足，就无法得到正确结果。因此，本次实验不仅加深了我对门限秘密共享原理的理解，也让我体会到密码协议在实际实现中需要同时关注安全性、正确性和工程细节。